

"People's feelings are memories that transcend time"
- Makise Kurisu

PEMROGRAMAN

July 25, 2026

Wisam Yassar Mahardika

CONTENTS

1 IOT	1
1.1 Pengertian IoT	1
1.1.1 Karakteristik Utama IoT	1
1.2 Sejarah IoT	1
1.3 Arsitektur IoT	2
1.4 Cara Kerja IoT	3
1.5 Komponen Utama IoT	3
1.6 Contoh Penerapan IoT	4
1.6.1 Smart Home	4
1.6.2 Smart Farming	4
1.7 Kelebihan & Tantangan IoT	4
1.7.1 Kelebihan IoT	4
1.7.2 Tantangan IoT	5
1.8 Kesimpulan	5
2 Algoritma & Flowchart	6
2.1 Pengertian Algoritma	6
2.1.1 Ciri-ciri Algoritma yang Baik	6
2.1.2 Cara Penyajian Algoritma	6
2.2 Flowchart	7
2.2.1 Jenis-jenis Flowchart	7
2.2.2 Struktur Dasar Algoritma/Flowchart	7
2.2.3 Kaidah Membuat Flowchart	8
2.3 Perbedaan Algoritma dan Flowchart	9

1 IOT

1.1 Pengertian IoT

Internet of Things (IoT) adalah konsep di mana benda-benda fisik sehari-hari dilengkapi dengan kemampuan untuk mengumpulkan, memproses, dan bertukar data melalui jaringan, tanpa memerlukan interaksi manusia secara langsung (*machine-to-machine* / M2M).

Inti dari IoT adalah menjembatani dunia fisik (*physical world*) dan dunia digital (*cyber world*) sehingga benda mati bisa "berpikir" sederhana, dan "bertindak".

Tujuan utama dari IoT adalah menciptakan lingkungan yang lebih cerdas (*smart environment*), efisien, dan otomatis, di mana data yang dikumpulkan oleh perangkat dapat dianalisis untuk menghasilkan wawasan (*insight*) yang bermanfaat, mendukung pengambilan keputusan, serta meningkatkan kenyamanan dan produktivitas manusia.

1.1.1 Karakteristik Utama IoT

- **Konektivitas (*Connectivity*):** Perangkat mampu terhubung ke internet atau jaringan lokal.
- **Sensing:** Memiliki sensor untuk mendeteksi kondisi lingkungan sekitar.
- **Otomatisasi (*Automation*):** Dapat bekerja dan mengambil tindakan tanpa campur tangan manusia secara langsung.
- **Pertukaran Data (*Data Exchange*):** Mampu mengirim dan menerima data secara *real-time*.
- **Interoperabilitas:** Dapat berkomunikasi dengan berbagai jenis perangkat dan platform yang berbeda.

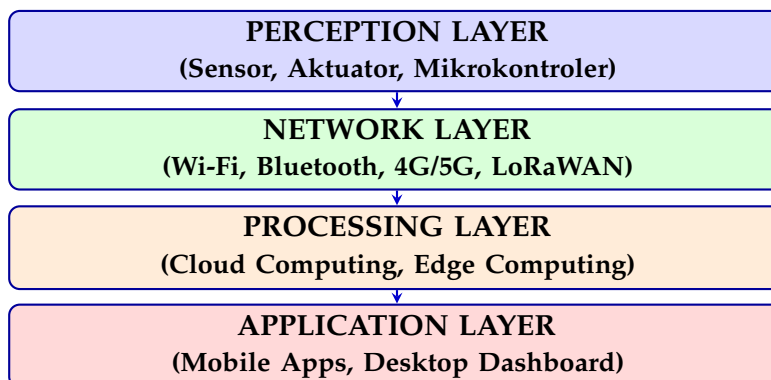
1.2 Sejarah IoT

Berikut adalah garis waktu (*timeline*) perkembangan konsep dan teknologi Internet of Things dari masa ke masa:



1.3 Arsitektur IoT

Arsitektur sistem IoT umumnya dibagi menjadi 4 lapisan (*layers*) utama:



Perception Layer: Lapisan fisik atau perangkat keras paling dasar yang bertugas mengumpulkan data dari lingkungan nyata (seperti panca indra manusia). Mengubah kondisi fisik menjadi data digital. *Contoh: Sensor, Aktuator, Mikrokontroler.*

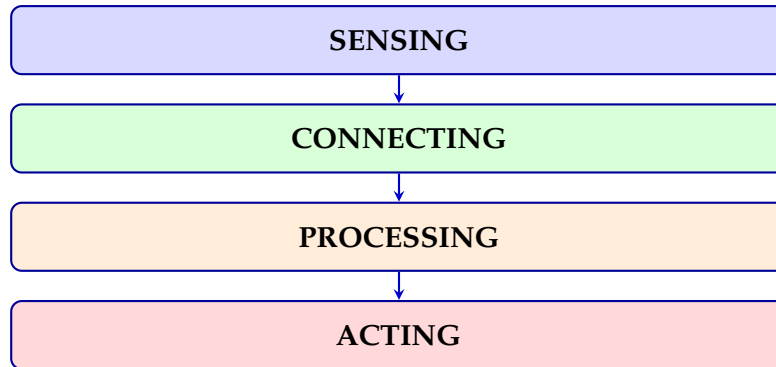
Network Layer: Lapisan infrastruktur konektivitas yang merutekan dan mentransmisikan data dari perception layer menuju server cloud atau pusat data. *Contoh: Wi-Fi, Bluetooth, GSM/4G/5G, LoRaWAN.*

Processing Layer: Lapisan "otak" sistem yang menyaring, menganalisis, dan mengolah data mentah menjadi informasi bermakna memanfaatkan teknologi *Cloud Computing* atau *Edge Computing*.

Application Layer: Lapisan teratas yang berfungsi sebagai antarmuka langsung (*User Interface*) bagi pengguna untuk memantau dan mengendalikan perangkat pintar secara *real-time*.

1.4 Cara Kerja IoT

Secara umum, alur kerja sistem IoT terbagi menjadi empat langkah berurutan:



1. **Sensing:** Sensor mendeteksi kondisi fisik lingkungan (suhu, kelembapan, cahaya, gerakan, posisi) dan mengubahnya menjadi sinyal digital.
2. **Connecting:** Data dikirimkan melalui jaringan komunikasi (Wi-Fi, Bluetooth, Seluler, MQTT) menuju gateway atau server.
3. **Processing:** Data diolah dan dianalisis pada cloud/edge computing untuk menghasilkan wawasan atau perintah.
4. **Acting:** Sistem memberikan notifikasi kepada pengguna atau menginstruksikan aktuator secara otomatis untuk melakukan tindakan fisik.

1.5 Komponen Utama IoT

Sistem IoT dibangun dari beberapa komponen kunci berikut:

- **Perangkat Sensor:** Mengubah parameter fisik menjadi sinyal elektrik.
 - *DHT11 / DHT22:* Mendeteksi suhu udara dan kelembapan lingkungan.
 - *Sensor Ultrasonik (HC-SR04):* Mengukur jarak objek berbasis pemantulan gelombang suara ultrasonik.
- **Aktuator:** Mengubah perintah elektrik menjadi gerakan atau aksi fisik.
 - *Solenoid:* Kumparan magnetik untuk menarik/mendorong komponen mekanis (contoh: Kunci pintu otomatis).
 - *Motor Servo:* Mengontrol gerakan putaran sudut secara presisi.
- **Mikrokontroler:** Chip pemroses ("otak") lokal perangkat.
 - *Arduino UNO:* Memproses logika dasar tanpa konektivitas jaringan terintegrasi.
 - *ESP32:* Chip dengan performa tinggi dilengkapi modul Wi-Fi dan Bluetooth internal.

- **Jaringan Komunikasi:** Media pengiriman data nirkabel (Wi-Fi, Bluetooth, LoRaWAN, Cellular).
- **Cloud Platform:** Layanan terpusat untuk penyimpanan dan analisis data besar (contoh: Microsoft Azure IoT, AWS IoT, Blynk).
- **Antarmuka Pengguna (*User Interface*):** Aplikasi mobile atau antarmuka web desktop untuk kontrol dan visualisasi data.

1.6 Contoh Penerapan IoT

1.6.1 Smart Home

Penerapan IoT pada perangkat rumah tangga seperti lampu, AC, kunci pintu, dan CCTV yang dikendalikan dan dipantau secara otomatis atau melalui smartphone.

- **Sensor/Objek:** Perangkat fisik terhubung sensor.
- **Penghubung:** Jaringan internet / Wi-Fi.
- **Pengontrol:** Smartphone atau *Voice Assistant*.

1.6.2 Smart Farming

Integrasi teknologi IoT untuk memantau, mengontrol, dan mengoptimalkan proses bercocok tanam (*precision agriculture*).

- Monitoring kelembapan tanah dan suhu udara secara *real-time*.
- Sistem penyiraman dan pemupukan otomatis.
- Penggunaan drone pertanian untuk analisis kesehatan tanaman.

1.7 Kelebihan & Tantangan IoT

1.7.1 Kelebihan IoT

1. **Efisiensi & Otomatisasi:** Mengurangi tugas manual dan rutin secara signifikan.
2. **Pengambilan Keputusan Berbasis Data:** Data *real-time* memungkinkan keputusan lebih akurat.
3. **Peningkatan Produktivitas:** Otomatisasi bisnis dan operasional industri.
4. **Kenyamanan Pengguna:** Pengendalian perangkat secara jarak jauh dari mana saja.
5. **Penghematan Biaya:** Mencegah kerusakan berat melalui *predictive maintenance*.

1.7.2 Tantangan IoT

1. **Keamanan Data (*Security*):** Risiko kebocoran data dan peretasan perangkat terhubung.
2. **Privasi Pengguna:** Potensi penyalahgunaan data pribadi.
3. **Keterbatasan Infrastruktur:** Membutuhkan koneksi internet yang stabil dan merata.
4. **Biaya Implementasi:** Biaya awal pengadaan sensor dan perangkat keras.
5. **Konsumsi Energi:** Kebutuhan konsumsi daya konstan dan manajemen baterai.

1.8 Kesimpulan

1. Internet of Things (IoT) adalah teknologi yang menghubungkan berbagai perangkat fisik ke internet untuk bertukar data secara mandiri tanpa interaksi manusia langsung.
2. Istilah IoT pertama kali dicetuskan oleh Kevin Ashton pada tahun 1999.
3. Arsitektur IoT terdiri atas 4 lapisan utama: *Perception Layer*, *Network Layer*, *Processing Layer*, dan *Application Layer*.
4. Keberhasilan ekosistem IoT bergantung pada integrasi harmonis antara sensor, aktuator, mikrokontroler, jaringan komunikasi, cloud platform, dan antarmuka pengguna.

2 ALGORITMA & FLOWCHART

Konsep Dasar, Simbol, Struktur, hingga Penerapannya.

2.1 Pengertian Algoritma

Algoritma adalah kumpulan langkah-langkah logis dan sistematis yang disusun secara berurutan untuk menyelesaikan suatu masalah atau mencapai tujuan tertentu. Setiap langkah dalam algoritma harus terdefinisi dengan jelas sehingga dapat dipahami dan dilaksanakan, baik oleh manusia maupun oleh komputer, tanpa menimbulkan makna ganda.

Kata algoritma berasal dari kata "Algoritmi", yaitu nama latin dari matematikawan Persia bernama Abu Ja'far Muhammad Ibn Musa Al-Khawarizmi yang menjadi tokoh dasar lahirnya algoritma modern.

Tujuan Algoritma:

- **Menyusun langkah penyelesaian masalah** secara terstruktur dan logis.
- **Mempermudah proses analisis dan perancangan program** sebelum implementasi penulisan kode.
- **Menjadi panduan universal** yang tidak bergantung pada sintaks bahasa pemrograman tertentu.
- **Meminimalkan kesalahan logika (*logic error*)** karena langkah dan kondisi dievaluasi secara jelas di awal.

2.1.1 Ciri-ciri Algoritma yang Baik

Menurut Donald E. Knuth, algoritma yang baik harus memenuhi kriteria berikut:

- **Finiteness (Terbatas):** Harus berhenti setelah melakukan sejumlah langkah tertentu dan memiliki akhir yang jelas (tidak *infinite loop*).
- **Definiteness (Jelas dan Tidak Ambigu):** Setiap langkah didefinisikan secara tegas tanpa penafsiran ganda.
- **Input:** Memiliki nol atau lebih data masukan sebelum atau selama proses.
- **Output:** Menghasilkan setidaknya satu keluaran dari hasil penyelesaian masalah.
- **Effectiveness (Efektif dan Efisien):** Langkahnya sederhana, wajar, dan hemat sumber daya.

2.1.2 Cara Penyajian Algoritma

Kalimat Deskriptif: Dituliskan dalam bentuk kalimat naratif menggunakan bahasa sehari-hari secara runtut. Mudah dipahami, namun kurang efektif untuk logika kompleks.

Pseudocode: Penulisan menggunakan campuran bahasa manusia dan struktur mirip bahasa pemrograman. Lebih terstruktur dan mudah diterjemahkan ke dalam kode.

Flowchart: Disajikan dalam bentuk diagram visual menggunakan simbol standar dan garis alir (*flow line*). Sangat memudahkan pemahaman visual.

2.2 Flowchart

Flowchart (diagram alir) adalah representasi grafis dari suatu algoritma yang menggambarkan urutan proses dan keputusan logis menggunakan simbol-simbol standar (seperti kotak, oval, dan belah ketupat) yang dihubungkan dengan garis alir.

2.2.1 Jenis-jenis Flowchart

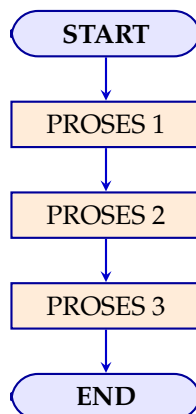
- **Flowchart Sistem:** Menggambarkan alur kerja sistem secara keseluruhan (hardware, software, dan data).
- **Flowchart Dokumen:** Menggambarkan alur dokumen/formulir antar unit dalam organisasi.
- **Flowchart Skematik:** Mirip flowchart sistem namun dilengkapi dengan ilustrasi gambar fisik perangkat.
- **Flowchart Program:** Menggambarkan urutan instruksi program secara rinci (paling sering digunakan di pemrograman dasar).
- **Flowchart Proses:** Menggambarkan proses kerja dalam industri atau proses produksi.

2.2.2 Struktur Dasar Algoritma/Flowchart

Terdapat tiga struktur dasar untuk menyusun alur flowchart:

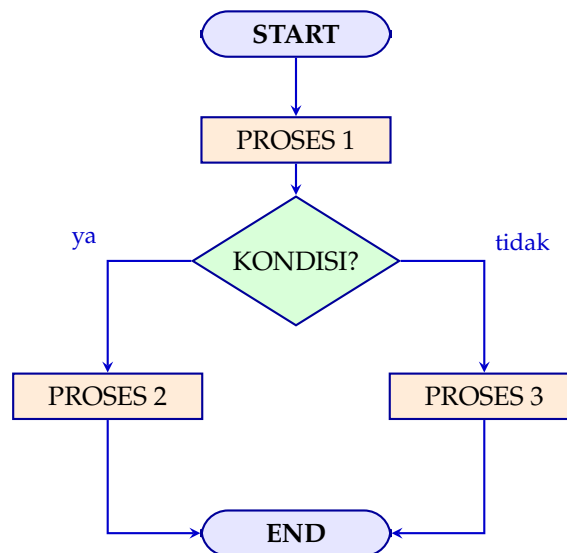
1. Sequence (Runtutan)

Instruksi dijalankan secara berurutan dari atas ke bawah, tanpa pengulangan atau percabangan.



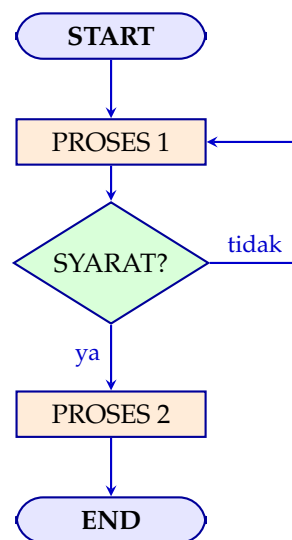
2. Selection (Percabangan/Branching)

Program mengambil salah satu dari beberapa jalur proses berdasarkan suatu kondisi (*IF-THEN-ELSE*).



3. Repetition (Perulangan/Looping)

Satu atau beberapa instruksi dijalankan berulang kali selama suatu kondisi masih terpenuhi.



2.2.3 Kaidah Membuat Flowchart

1. Digambar dari atas ke bawah dan dari kiri ke kanan secara konsisten.
2. Hanya memiliki satu titik awal (*start*) dan satu titik akhir (*end*).
3. Menggunakan simbol standar (ANSI/ISO) sesuai fungsinya.
4. Gunakan kalimat aktif dan singkat pada keterangan langkah.
5. Simbol *decision* hanya boleh memiliki satu alur masuk, namun boleh memiliki beberapa alur keluar.
6. Hindari alur saling silang; gunakan *connector* jika diperlukan.
7. Jaga jarak antar simbol secara proporsional agar mudah dibaca.

2.3 Perbedaan Algoritma dan Flowchart

Aspek	Algoritma	Flowchart
Sifat	Abstrak dan fleksibel.	Konkret dan visual.
Penyajian	Teks deskriptif atau <i>pseudocode</i> .	Diagram dengan simbol standar.
Detail	Bisa sangat umum maupun rinci.	Cenderung rinci karena tiap alur butuh simbol.
Keterbacaan	Butuh penalaran bacaan teks logis.	Lebih cepat dipahami sekilas secara visual.